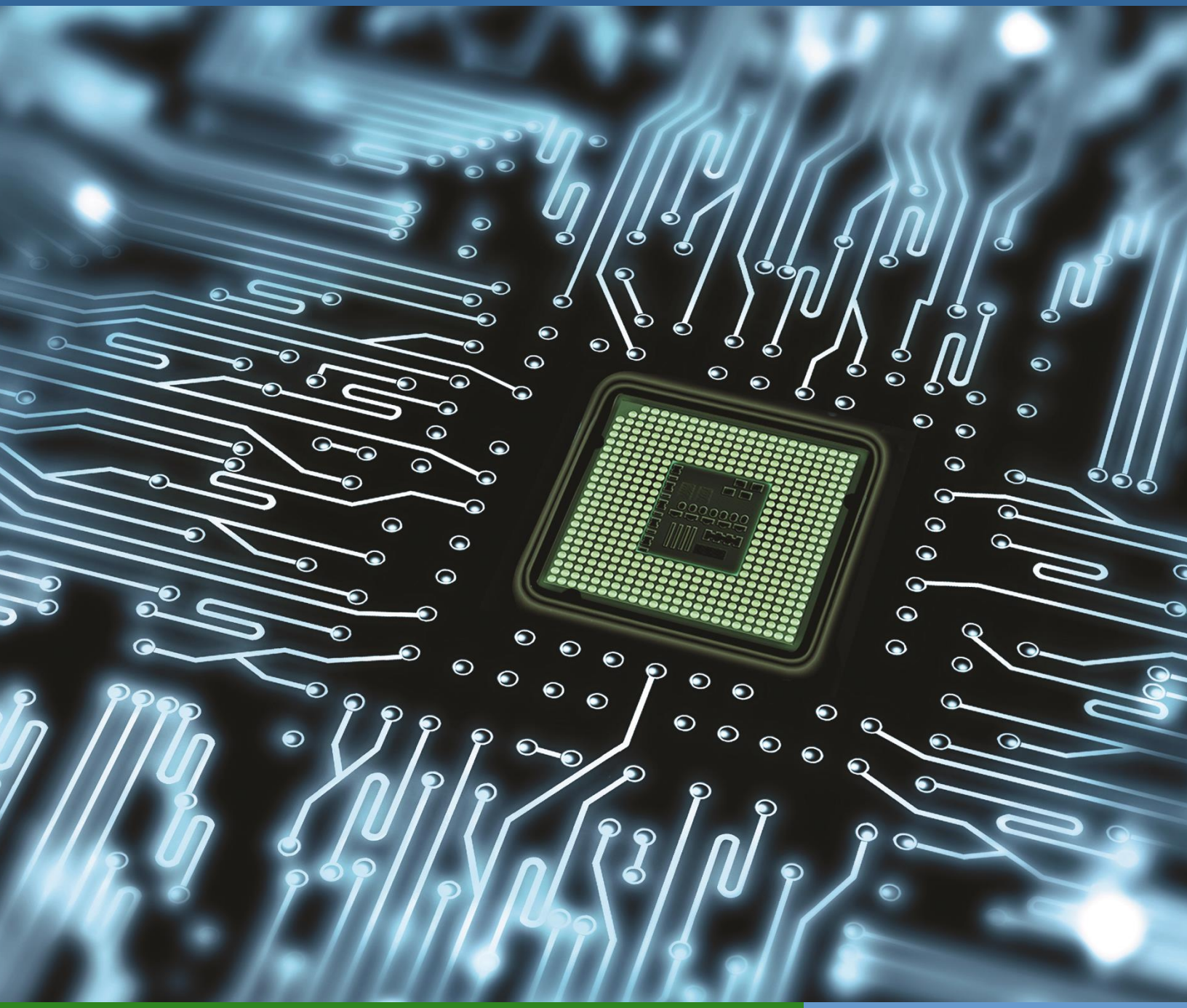
Semester Project Report



Members:

- Ahtisham Saleem(NIM-BSEE-2021-19)
- M. Rafique(NIM-BSEE-2021-41)
- M. Makki(NIM-BSEE-2021-23)

Instructor:

Mam Naureen Shaukat

TABLE OF CONTENTS

1. Introduction

1.1. Objective
1.2. Introduction to ALU
1.3. Explanation
1.4. Component used
- OR GATE IC
- AND GATE IC
- NOT GATE IC
- XOR GATE IC
- 4-BIT Binary Adder
- 4*1 Multiplexer

1.5. Functionality of ALU
- ALU operations

1.6. Logic diagram
1.7. Hardware

2. Software Part

2.1. Software used
- Verilog HDL
- ModelSim

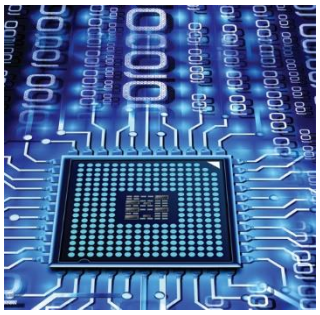
2.2. Verilog Code

3. Conclusion

1. Introduction

1.1.Objective

A four-bit arithmetic and logic unit's (ALU) circuit is to be designed on hardware and in Verilog HDL. It should have two inputs of 4-bits each and be able to perform one operation at a time among the following arithmetic and logical operations:



1. Adding two numbers
2. Subtracting two numbers.
3. Taking the bit-wise AND of two numbers.
4. Taking the bit-wise OR of two numbers.
5. Taking the bit-wise NOT of two numbers. Students should demonstrate the output through 4 LEDs

1.2. Introduction to ALU

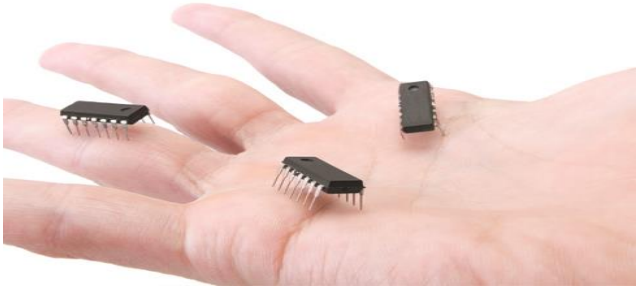
An Arithmetic Logic Unit (ALU) is a digital circuit used to perform basic arithmetic and logical operations. It is the fundamental building block of the central processing unit (CPU) of a computer. An ALU can add, subtract, multiply, divide, and perform logical operations such as AND, OR, NOT, and XOR. ALUs are also used to store and manipulate data in memory.

1.3. Explantion

In this semester project we are required to use 4 bit ALU so that it can be able to perform operations on 4 bit binary numbers. Also we will use two MUX IC's which will control the required the operations of the ALU.

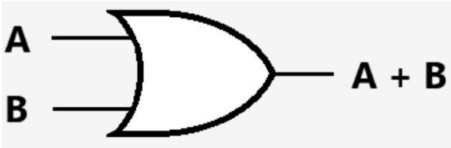
1.4. Component Used

Following components (IC's) are being used in this project.



- **OR GATE IC**

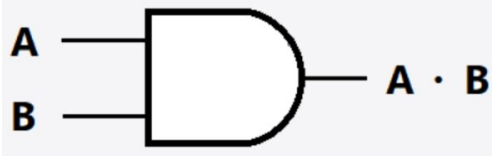
  A basic gate needed in digital electronics is the OR gate. It displays the result as the result of adding two inputs. The OR Gate's IC number is 7432.



  A digital logic gate called an OR gate combines both inputs and produces a single output. The output of an OR gate is also high if one of its inputs is high. A low output results from two low inputs, whereas a high output results from two high inputs. This is total function of an OR gate.

- **AND GATE IC**

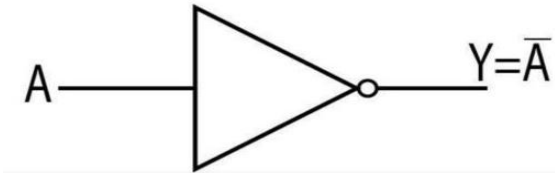
  A basic digital logic gate is an AND gate. By multiplying the two inputs, it produces the output. AND Gate's IC number is 7408.



  Two inputs and one output make up an AND gate. The output of an AND gate is low if one of its inputs is low. The output will be high if the inputs are high.

- **NOT GATE IC**

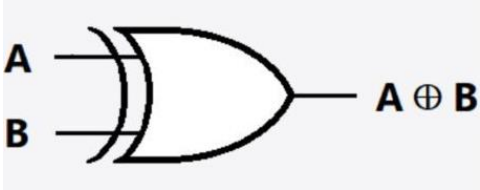
  In digital electronics, a NOT gate is referred to as an inverter. because the NOT gate inverts the input and output. NOT Gate's IC number is 7404.



  The output of an inverter circuit represents to the input's inversion. Its overall purpose is to invert the applied input signal. The output will have a high value if the input is low, and vice versa. A NOT gate is very simple to build and only needs a few tools to be designed. As a result, only a little power is required to run the NOT gate.

- **XOR GATE IC**

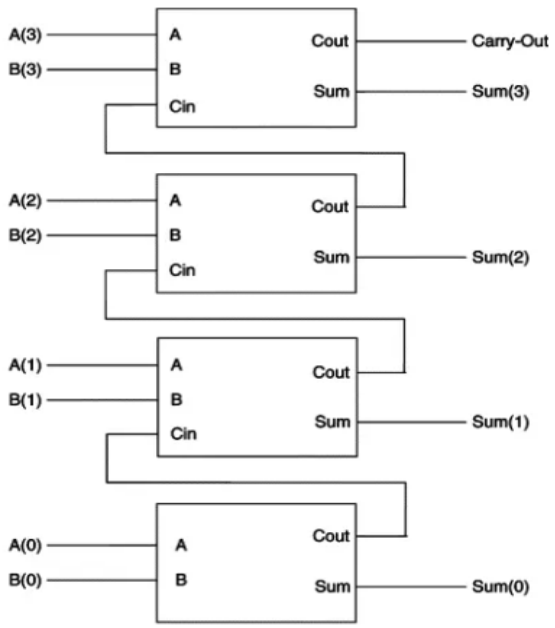
  In digital electronics, the X-OR gate is commonly referred to as an exclusive OR or Ex-OR. The X-OR Gate's IC number is 7486.



  The X-OR gate's output is high if its inputs each contain an odd number of ones. The output is low if both inputs have the same value. Because X-OR gates are often unavailable, we can create the X-OR gate using basic gates and universal gates. Therefore, a circuit with the same functionality can be created using other available gates.

- **4 BIT BINARY FULL ADDER**

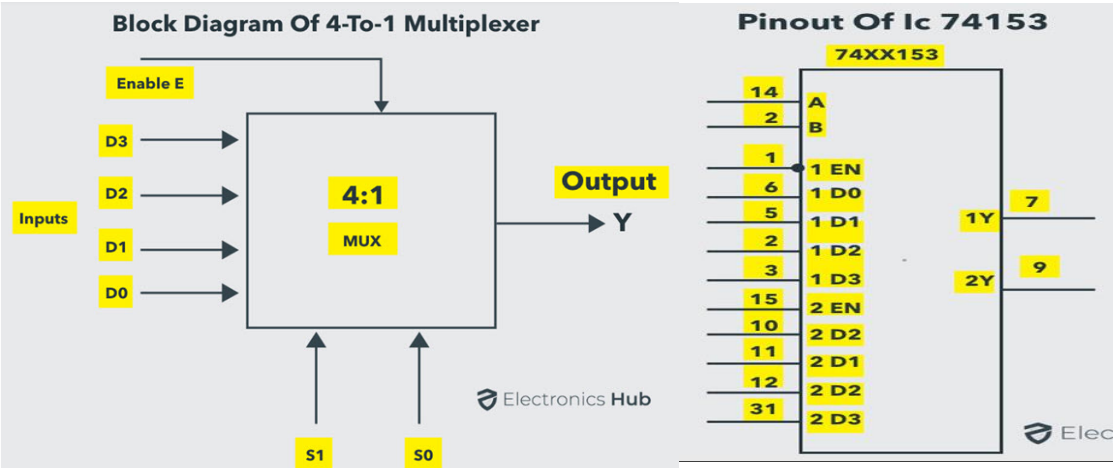
A four bit full adder may add four input bits. T he definition of a four-bit full adder: "A four-bit full adder is a logical circuit that takes carry and two expressions with four bits as its inputs and as a result shows the Four bits output in addition to the carry at output terminals."
The XOR Gate, AND Gate, and OR Gate combine up to form the Circuit of Four Bit Full Adder.



A binary adder-subtractor is a device used in digital circuits that can add and subtract binary numbers in a single circuit. Depending on the binary value the control signal provides, the operation is carried out. It is one of the components of Arithmetic Logic Unit.

- **4x1 multiplexer**

A 4-to-1 multiplexer has a single output line (Y), two select lines (S0 and S1), and four data input lines (D0 to D3). The four input lines are chosen by the select lines S0 and S1 to connect to the output line. The block diagram of a 4-to-1 multiplexer is shown in the picture below. The multiplexer decodes the input through a select line.



Usually, two 4-to-1 multiplexers will be contained within a single electrical device (IC) when this type of multiplexer is available. The double 4-to-1 line multiplexer IC 74153 is the most widely used and most widely available 4-to-1 line multiplexer. It comprises of two 4-to-1 multiplexers that are equal. To turn on or off the various multiplexers, it has two separate enable or pulse inputs. The Select lines, however, are shared by both Multiplexers.

Typically, a multiplexer with a lot of inputs can be built by cascading two or more multiplexer ICs using the enable input or strobe. The inputs to each multiplier are provided separately. The pinout of the IC74153 is shown in the figure above.

Some other components used in the completion of the project are:

- Trainner
- Wire
- Breadboard

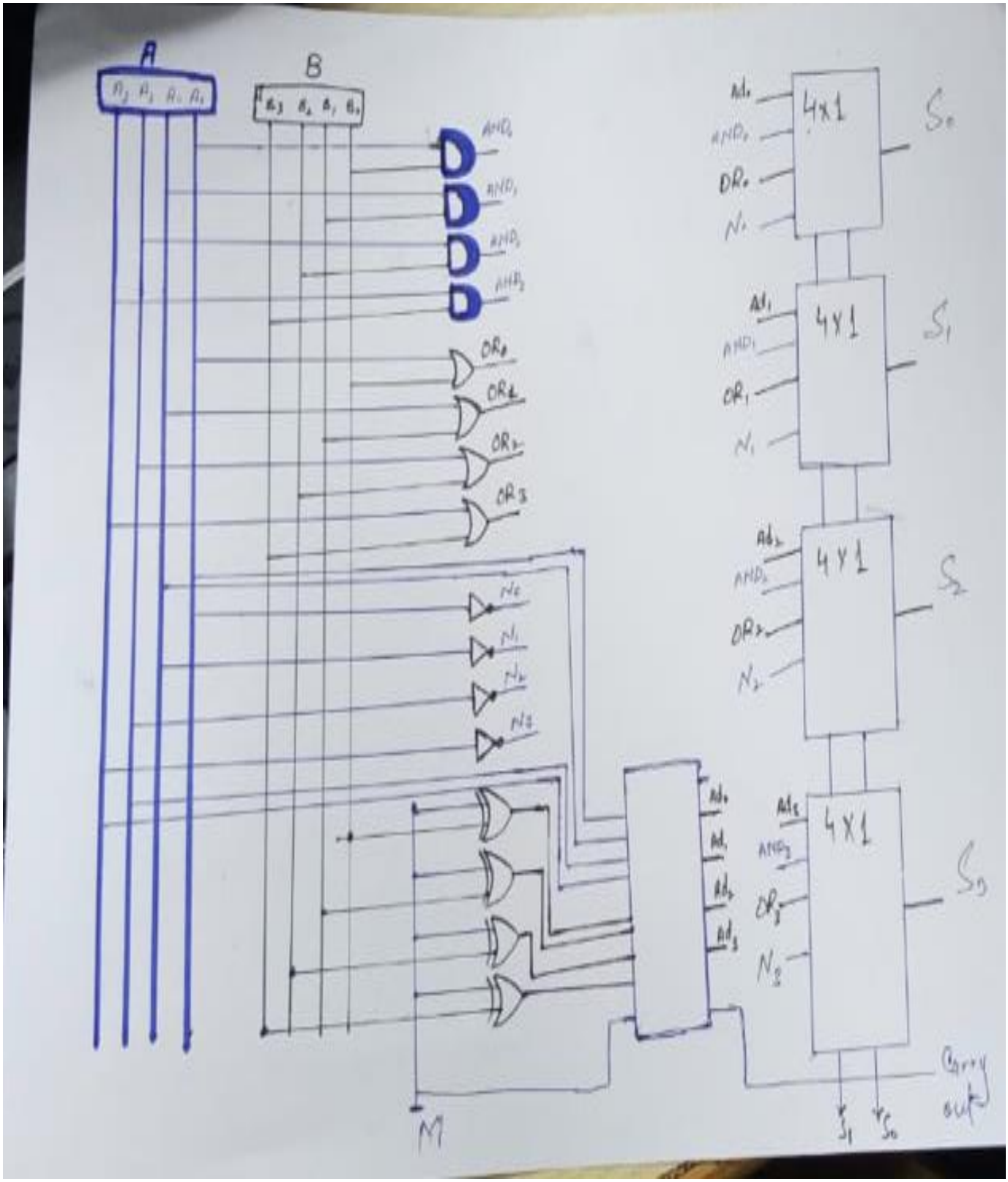
1.4. Functionality of ALU

As discussed earlier ALU is the building block for any system and here in this project we perform five possible operations by giving it specific inputs.

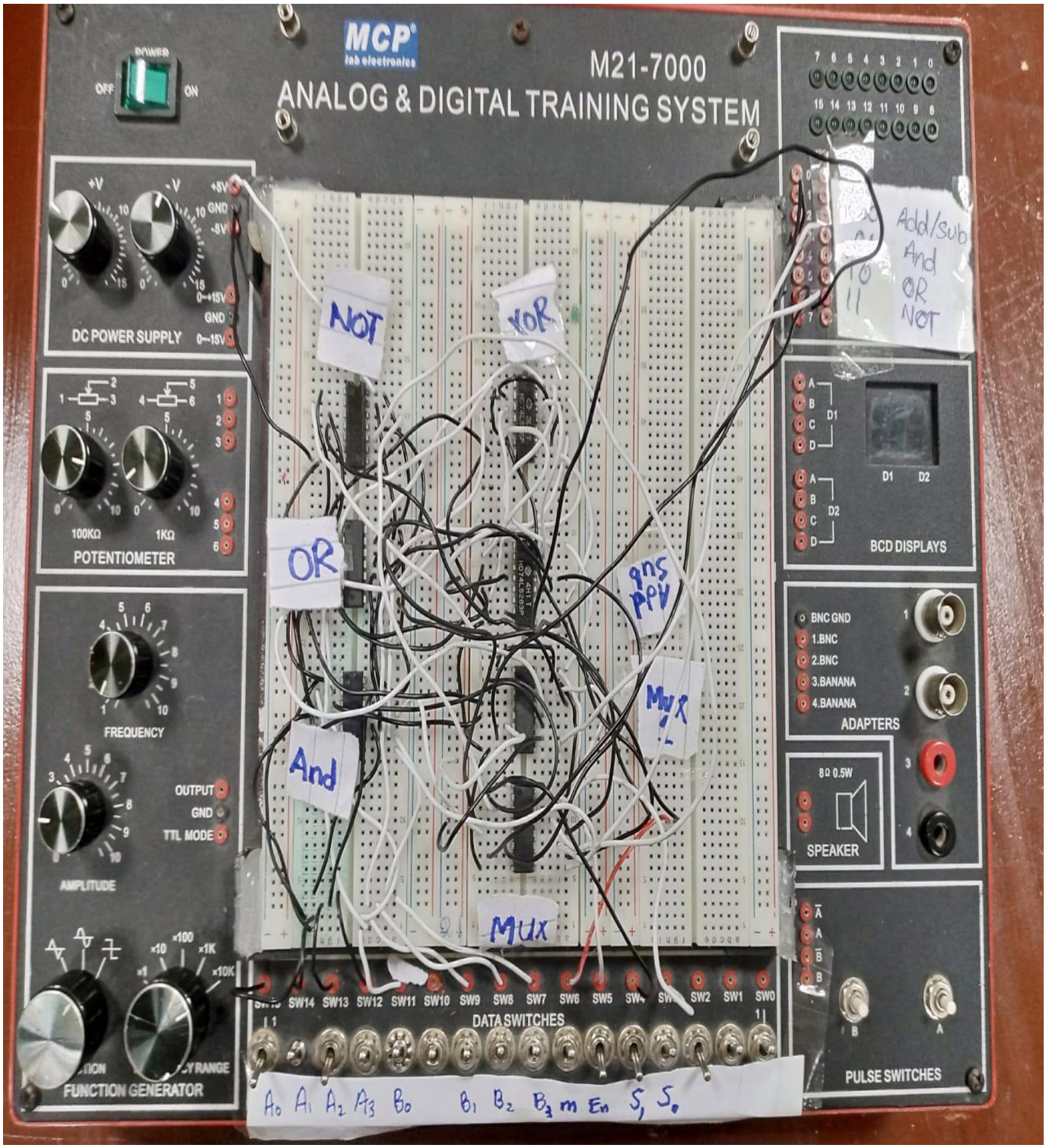
ALU OPERATIONS

- *If we provide 00 at selection line then addition or subtraction will be performed.*
- *If we provide 01 at selection line then OR operation will be performed.*
- *If we provide 10 at selection line then AND operation will be performed.*
- *If we provide 11 at selection line then NOT operation will be performed.*

1.5. Logic diagram



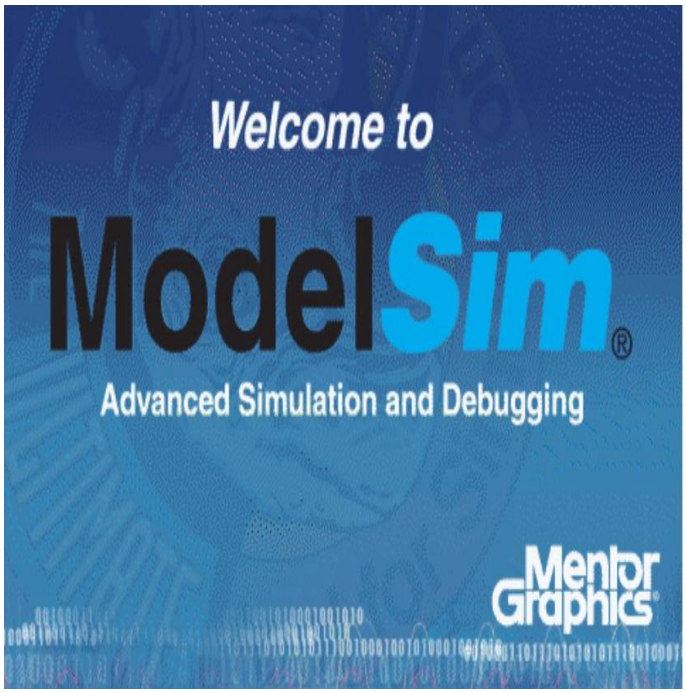
1.6. Hardware


MCP
lab electronics
M21-7000
ANALOG & DIGITAL TRAINING SYSTEM
POWER
OFF
ON
DC POWER SUPPLY
POTENTIOMETER
FREQUENCY
AMPLITUDE
OUTPUT
GND
TTL MODE
FUNCTION GENERATOR
BCD DISPLAYS
ADAPTERS
SPEAKER
DATA SWITCHES
PULSE SWITCHES
NOT
XOR
OR
And
Mux
Add/Sub
And
OR
NOT

2.Software Part

2.1. Software Used

The software used for the programming of the ALU was Verilog HDL (ModelSim). Verilog is a HARDWARE DESCRIPTION LANGUAGE (HDL). ModelSim is a program created by Mentor Graphics used for simulating your VHDL and Verilog designs. It is the most widely use simulation program in business and education. It is a language used for describing a digital system like a network switch or a microprocessor or a memory or a flip−flop. It means, by using Verilog HDL we can describe any digital hardware at any level.



Verilog

In contrast to "regular" programming languages like C or Python, Verilog is a language for describing hardware. It is a method of describing digital logic that does not involve integrating chips or gates in a schematic, as was once the case (either physically or virtually).

ModelSim

Siemens' ModelSim is a multi-language environment with a built-in C debugger for simulating hardware description languages including VHDL, Verilog, and SystemC. ModelSim was previously developed by Mentor Graphics. ModelSim can be used independently or in conjunction with Intel Quartus Prime, PSIM, and other simulation software.

2.2. Verilog Code

```
module project(out,select,a,b,cin,sum,cout,bor,diff,m);
  input m,cin;
  input [3:0] a,b;
  input  [1:0] select;
  output reg[3:0] out,cout,bor,diff,sum;
always@(a or b or cin)
begin
if(select==00)
  begin
 if (m==0)
  begin
      //FULL ADDER 1
      sum[0]=a[0]^b[0]^cin;
      cout[0]=(a[0]&b[0])|((a[0]^b[0])&cin);

     //FULL ADDER 2
      sum[1]=a[1]^b[1]^cout[0];
     cout[1]=(a[1]&b[1])|((a[1]^b[1])&cout[0]);

    //FULL ADDER 3
      sum[2]=a[2]^b[2]^cout[1];
      cout[2]=(a[2]&b[2])|((a[2]^b[2])&cout[1]);

   //FULL ADDER 4
       sum[3]=a[3]^b[3]^cout[2];
       cout[3]=(a[3]&b[3])|((a[3]^b[3])&cout[2]);
     end
     if (m==1)
      begin
      //FULL ADDER 1
      diff[0]=a[0]^b[0]^cin;
      bor[0]=(a[0]&b[0])|((a[0]^b[0])&(~cin));

     //FULL ADDER 2
      diff[1]=a[1]^b[1]^cout[0];
```

```
      bor[1]=(a[1]&b[1])|((a[1]^b[1])&(~cout[0]));

    //FULL ADDER 3
      diff[2]=a[2]^b[2]^cout[1];
      bor[2]=(a[2]&b[2])|((a[2]^b[2])&(~cout[1]));

   //FULL ADDER 4
       diff[3]=a[3]^b[3]^cout[2];
       bor[3]=(a[3]&b[3])|((a[3]^b[3])&(~cout[2]));
     end
     end
   else if(select==01)
     begin
       out=a&b;
     end
     else if(select==10)
       begin
       out=a|b;
     end
       else if(select==11)
         begin
         out=!a;
       end
 end
endmodule
```

3.Conclusion

This project proved very helpful for us to acquire practical knowledge of digital logic design. 4-bit binary ALU is capable of performing arithmetic and logical operation on 4 bit binary numbers. All the operations were performed by applying different condition and giving different inputs. All the operations were implemented on hardware and then their verification was confirmed by performing same operations on Verilog HDL. This project provided an opportunity to explore technical aspects of the subject, hands-on training, and practical application of ALU. The learning outcomes from this project will help us to apply our best knowledge to the upcoming projects and tasks in the field of electrical engineering.